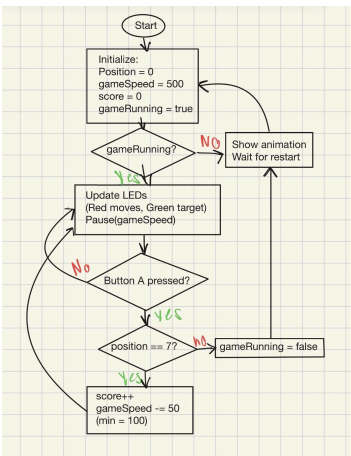
Problem Statement

This lab was about building a reaction-based game running in a simulated microcontroller environment. Basically, you've got a circle of LEDs, one moves around, while another stays fixed as the target. Your job is to press the button exactly when the moving LED lines up with the target. The program handles real-time input, conditional logic, and keeps the system dynamic.

Diagram



Flow Description

Here's how it works:

- Start program
- Initialize variables (position, speed, score, game state)
- Continuously update LED positions
- Monitor for button input
- If the button is pressed at the correct time → score increases and the game speeds up
- If the timing is incorrect → game ends
- After the game ends, the player can restart

Equations / Variables

The following variables were used:

- position = 0
- gameSpeed = 500 (initial delay in milliseconds)
- score = 0
- gameRunning = true

Logic:

- If position == 7, that is the target (correct timing)
- If position == 10, reset position to 0
- The game speed has a minimum limit of 100 milliseconds

Updates:

- position = position + 1
- If position == 10, then position = 0

Speed Adjustments:

- Correct timing: gameSpeed = gameSpeed - 50
- If gameSpeed < 100, then gameSpeed = 100

Game Rules:

- If the button is pressed when position == 7:
- → score = score + 1, and the game speeds up
- If pressed at the wrong time:
- → gameRunning = false (game ends)

Discussion

We built a reaction-based game using LEDs. The setup consists of ten LEDs arranged in a ring, where a red LED continuously moves around while a green LED stays fixed as the target. The goal is to press the button at the exact moment when the red LED aligns with the green LED.

At first, the game ran in a constant loop, and after each correct press, the delay decreased to increase difficulty. However, this caused a problem—once the delay reached zero or became negative, the game stopped working properly. This showed how important it is to handle edge cases in timing-based systems.

To fix this issue, we introduced a minimum speed limit so the delay never drops below a stable value. We also added a gameRunning variable to control whether the game is active or over, which helped manage the program flow more effectively and allowed the game to restart properly.

Adding a scoring system made the game more engaging. Each correct press increases the score, and sound effects provide feedback for both successful and incorrect inputs, improving the overall user experience.

One of the main challenges was maintaining a clean program flow in an event-driven system. Our initial version used nested loops, which made the behavior harder to control. By separating the game state from input handling, the program became more stable and easier to follow.

Overall, this lab helped reinforce concepts such as event-driven programming, real-time logic, and loop control. The final version of the game works reliably, fixes earlier issues, and provides

a more interactive experience. In the future, the game could be improved by randomizing the target LED, adding multiple difficulty levels, or incorporating additional sensors.